Project - High Level Design

on

ProjectTitle

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Avanish Bairagi	EN22CS301241
2	Bhavya Jain	EN22CS301268
3	Cheshta Patidar	EN22CS301293
4	Devyani Choudhary	EN22CS301328
5	Dhruv Rai	EN22CS3013

Group Name: G01D3

Project Number: 01

Industry Mentor Name: Mr. Suraj Nayak

University Mentor Name: Prof. Vineeta rathore

Academic Yea: 2025-2026

Table of Contents

1. Introduction

This document presents the High-Level Design (HLD) of the project titled **"Chat with Logistics Documents using Retrieval-Augmented Generation (RAG)"**, developed as part of the Datagami Skill Based Course at Medicaps University.

The system is designed to enable logistics professionals to interact with domain-specific documents such as invoices, shipment reports, delivery records, and operational manuals using natural language queries. By leveraging Retrieval-Augmented Generation (RAG), vector embeddings, similarity search mechanisms, and Large Language Models (LLMs), the system generates accurate, context-aware, and grounded responses strictly based on the uploaded documents.

The primary objective of this system is to improve information retrieval efficiency, reduce manual document scanning, and enhance decision-making within logistics workflows.

This document describes the overall architecture, system components, data design, and major technical considerations of the proposed solution.

1.1 Scope of the Document

The scope of this document includes:

- High-level architecture of the RAG-based system
- Application design and workflow description
- Component-level system breakdown
- API catalogue overview
- Data model and data storage strategy
- Data retention and migration strategy

This document focuses on the architectural and structural design aspects of the system. It does not include:

- Detailed source code implementation
- UI wireframe designs
- Deployment configuration details
- Infrastructure provisioning scripts

The HLD serves as a blueprint for understanding the system structure and guiding further development and deployment phases.

1.2 Intended Audience

This document is intended for:

- Academic evaluators and university mentors
- Industry mentors and technical reviewers
- Developers involved in implementation
- System architects and designers
- Project stakeholders

It provides sufficient technical insight for understanding the architecture while remaining accessible to non-implementation stakeholders.

1.3 System Overview

The proposed system follows a **Retrieval-Augmented Generation (RAG)** architecture to enable intelligent querying of logistics documents.

The overall system workflow is divided into two primary pipelines:

1. Document Ingestion Pipeline

- Users upload logistics documents through the Streamlit-based interface.
- The system extracts text from PDF documents.
- Extracted text is segmented into smaller chunks.
- Each chunk is converted into a high-dimensional vector embedding using an embedding model.
- The embeddings are stored in a FAISS Vector Database.
- Document metadata is stored for efficient filtering and retrieval.

2. Query Processing Pipeline

- The user submits a natural language query.
- The query is converted into a vector embedding.
- The system performs a similarity search in the FAISS vector database.
- Relevant document chunks are retrieved.
- Retrieved context is sent to the Groq API (LLama-3 model) along with the user query.
- The LLM generates a grounded response.
- The response is displayed to the user via the Streamlit UI.

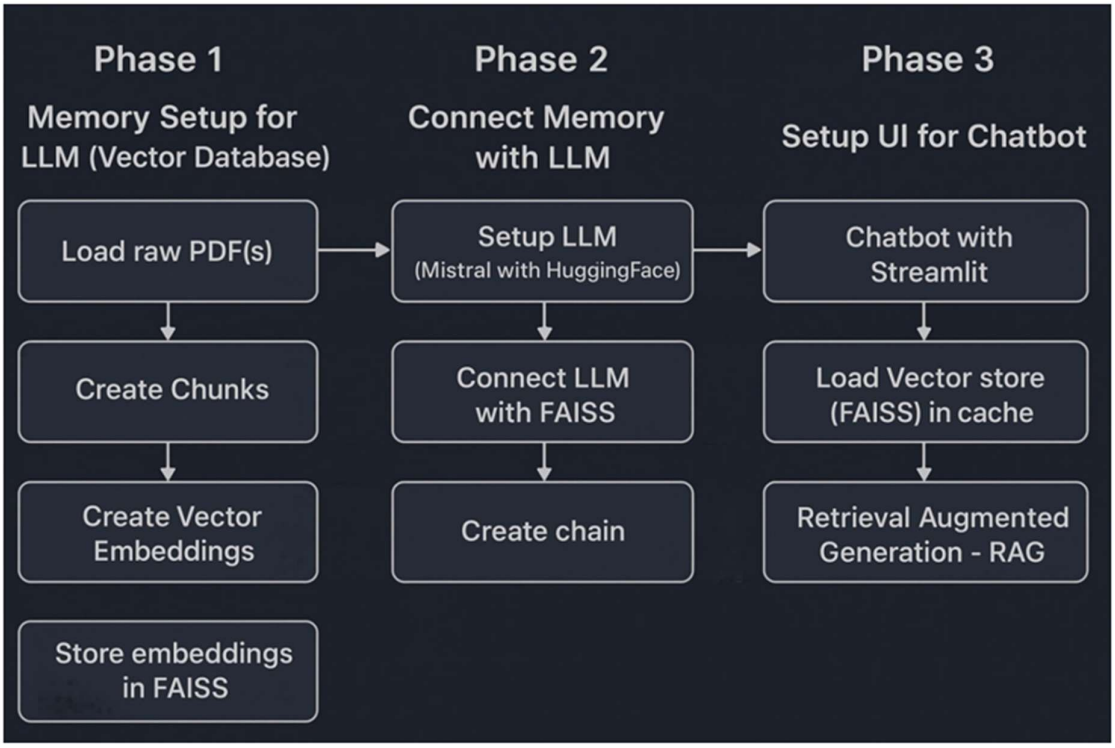


Figure 1.1 - Architecture Diagram

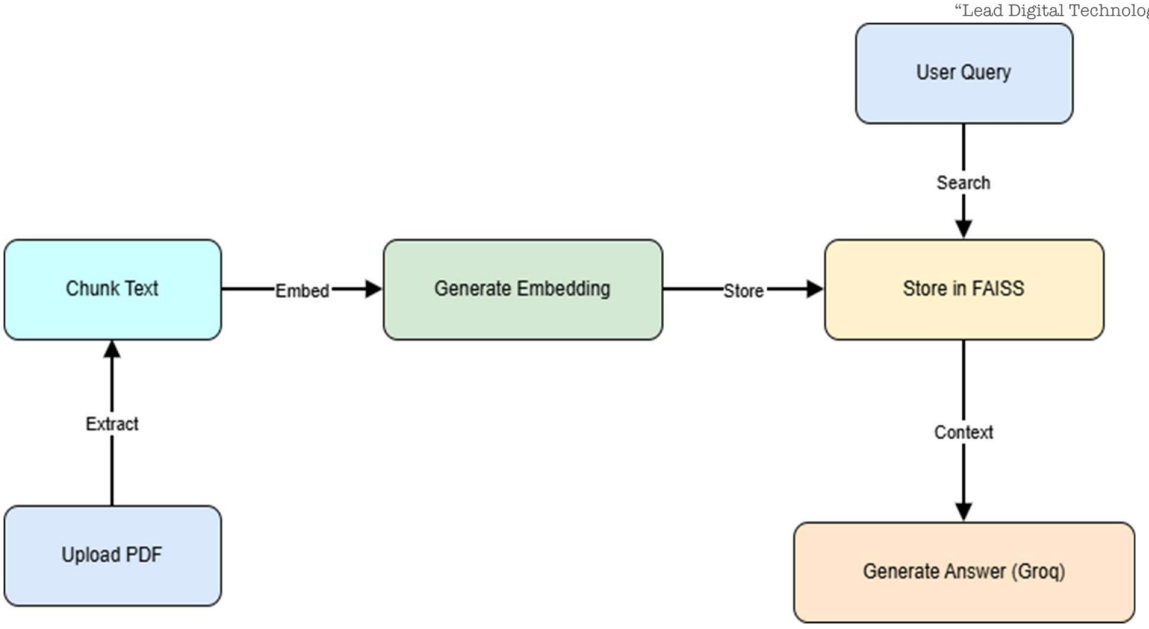


Figure 1.2 - High Level Architecture

This architecture ensures:

- Context-aware answer generation
- Reduced hallucination through retrieval grounding
- Fast similarity-based search
- Scalable and modular system design

2. System Design

This section describes the architectural structure, workflows, components, and design considerations of the proposed Retrieval-Augmented Generation (RAG)-based logistics document intelligence system.

2.1 Application Design

The proposed system follows a **layered and modular architecture** to ensure scalability, flexibility, and maintainability.

Architecture Overview

The application is structured into the following logical layers:

1. Presentation Layer

- Streamlit-based User Interface
- Document Upload Module
- Chat Interface
- Document Management Dashboard

This layer enables user interaction with the system.

2. Application Layer (Core RAG Pipeline)

This layer performs the main processing tasks:

- Document Processing Service (Text extraction and chunking)
- Embedding Generation Service
- Similarity Search Service
- LLM Orchestration Service
- Metadata Management Service

3. Data Layer

- FAISS Vector Database (stores embeddings)
- Relational Database (stores metadata and session data)
- File Storage (stores uploaded PDFs)

4. External Services Layer

- Embedding Model API
- Groq API (LLama-3) for response generation

2.2 Process Flow

The system consists of two primary workflows:

A. Document Ingestion Flow

1. User uploads document through UI.
2. Backend extracts text from PDF.
3. Text is cleaned and divided into chunks.
4. Chunks are converted into vector embeddings.
5. Embeddings are stored in FAISS.
6. Metadata is stored in relational database.
7. Document marked as successfully indexed.

B. Query Processing Flow

1. User submits query through chat interface.

2. Query is converted into embedding.
3. FAISS performs similarity search.
4. Top-K relevant chunks are retrieved.
5. Retrieved context + query sent to Groq API.
6. LLM generates response.
7. Answer displayed to user.

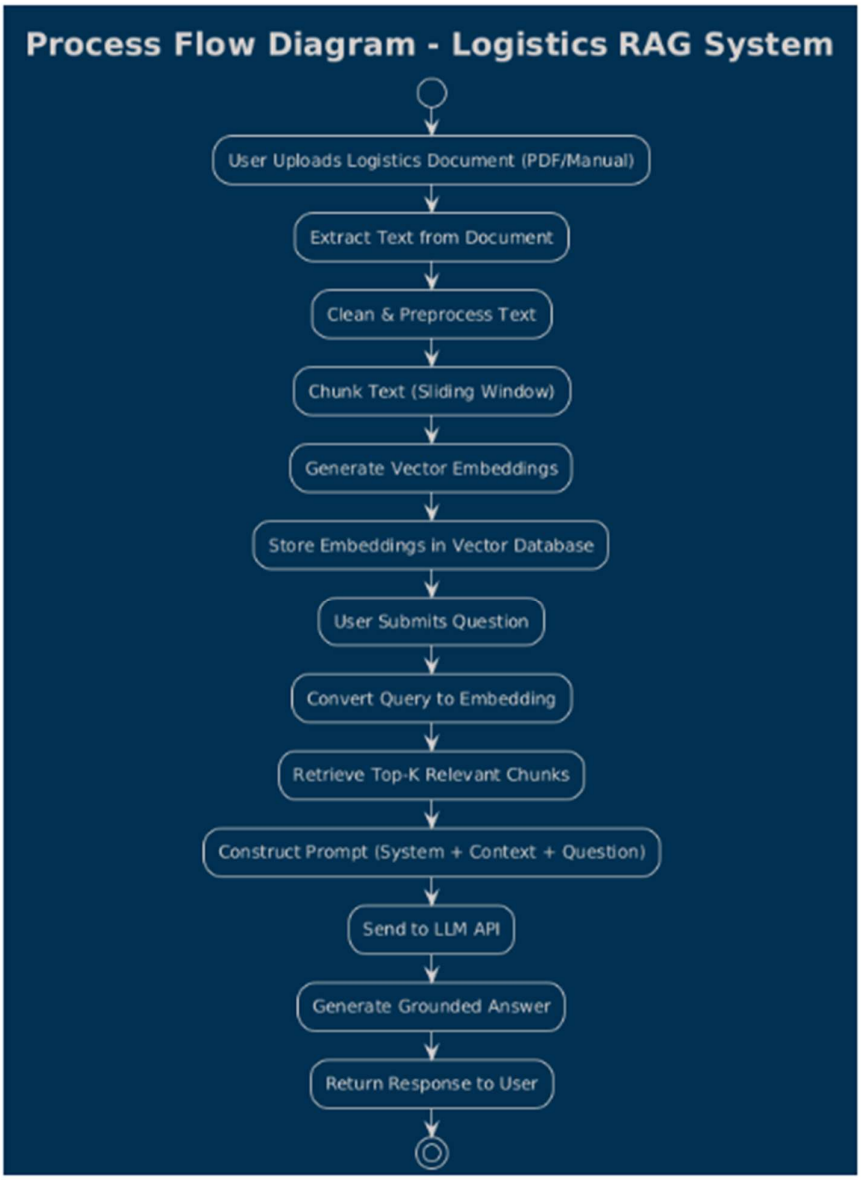


Figure 2.2.1 - Process Flow Diagram

The system processes uploaded logistics documents by extracting text, chunking it, and converting it into vector embeddings stored in a vector database. When a user submits a query, the system retrieves the most relevant document chunks using similarity search. These retrieved chunks are combined with the query and sent to the LLM to generate a context-aware, grounded response.

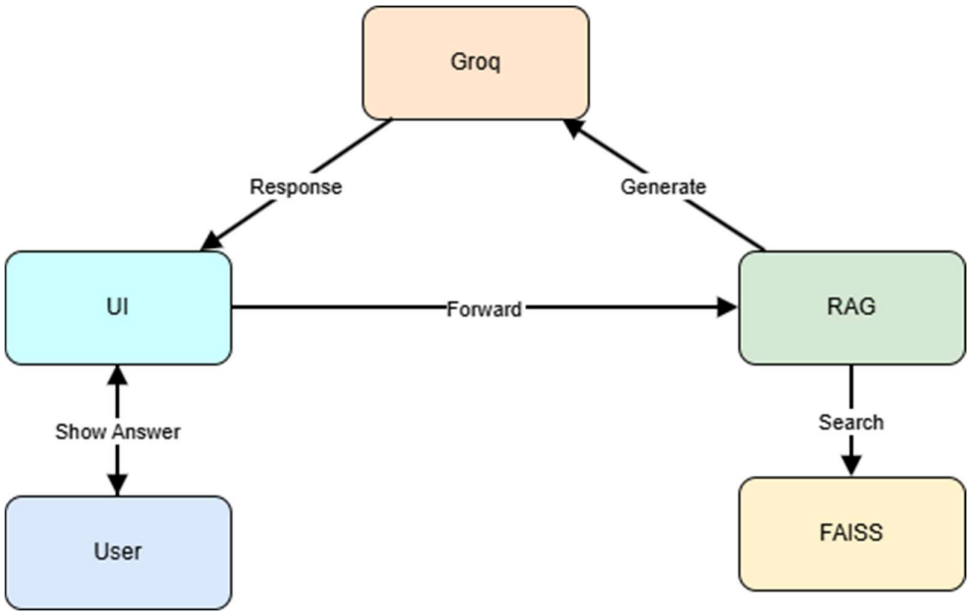


Figure 2.2.2 - Sequence Diagram

When the user submits a query, the frontend sends it to the backend, where it is converted into an embedding. The system retrieves the most relevant document chunks from the vector database and constructs a prompt with the query and context. The LLM generates a grounded response, which is returned to the user.

2.3 Information Flow

The system handles two main information streams:

Document Information Flow

PDF File
→ Text Extraction
→ Chunking
→ Embedding Generation
→ Vector Storage
→ Metadata Storage

Query Information Flow

User Query
→ Query Embedding

→ Similarity Search
→ Context Retrieval
→ Prompt Construction
→ LLM Response
→ Display to User

2.3 Information Flow

The system handles two main information streams:

Document Information Flow

PDF File
→ Text Extraction

→ Chunking

→ Embedding Generation
→ Vector Storage
→ Metadata Storage

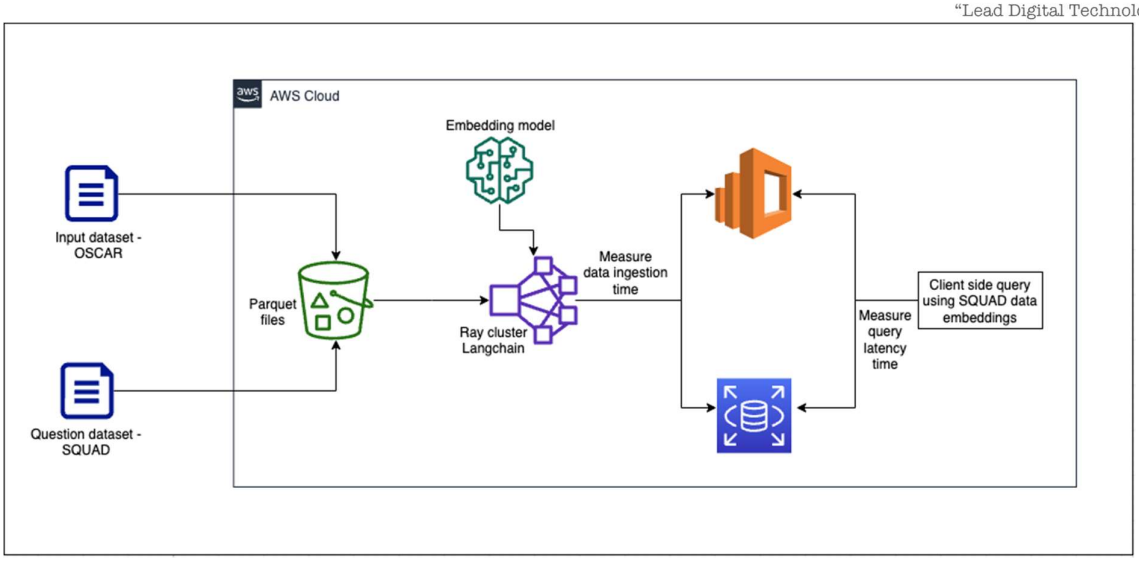


Figure 2.3.1 - Query flow diagram

Query Information Flow

User Query
→ Query Embedding
→ Similarity Search
→ Context Retrieval
→ Prompt Construction
→ LLM Response
→ Display to User

This separation ensures modularity between ingestion and retrieval pipelines.

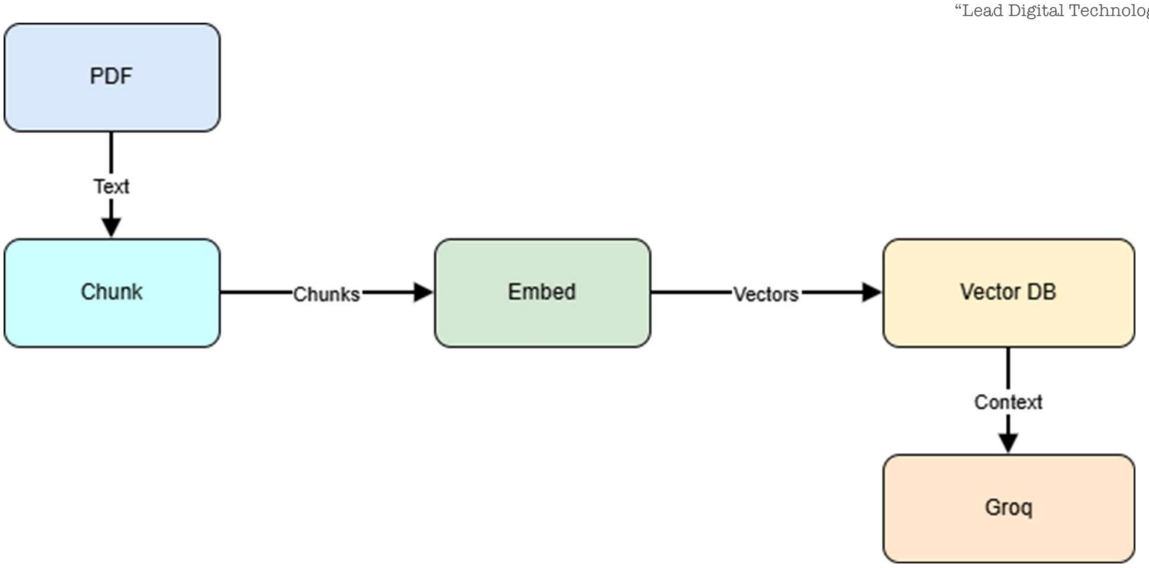


Figure2.3.2 - Data Flow Diagram (DFD – Level 1)

In Level 1 DFD, the system is divided into two main processes: document processing and query processing. Uploaded documents are stored and converted into embeddings in the vector database. When a user submits a query, relevant data is retrieved from the vector database to generate a grounded response.

2.4 Components Design

The system consists of the following major components:

1. User Interface Component

- Handles document upload
- Displays chat responses
- Provides document management dashboard

2. Document Processing Component

- Extracts text from PDFs
- Cleans and normalizes text
- Splits text into chunks

3. Embedding Component

- Converts text chunks into vector embeddings
- Handles batch embedding generation

4. Vector Database Component (FAISS)

- Stores embeddings
- Performs similarity search
- Retrieves top-K relevant chunks

5. LLM Orchestration Component

- Constructs prompt with retrieved context
- Calls Groq API (LLama-3)
- Returns grounded response

6. Metadata Management Component

- Stores document details (Shipment ID, Date, Carrier)
- Enables filtered retrieval

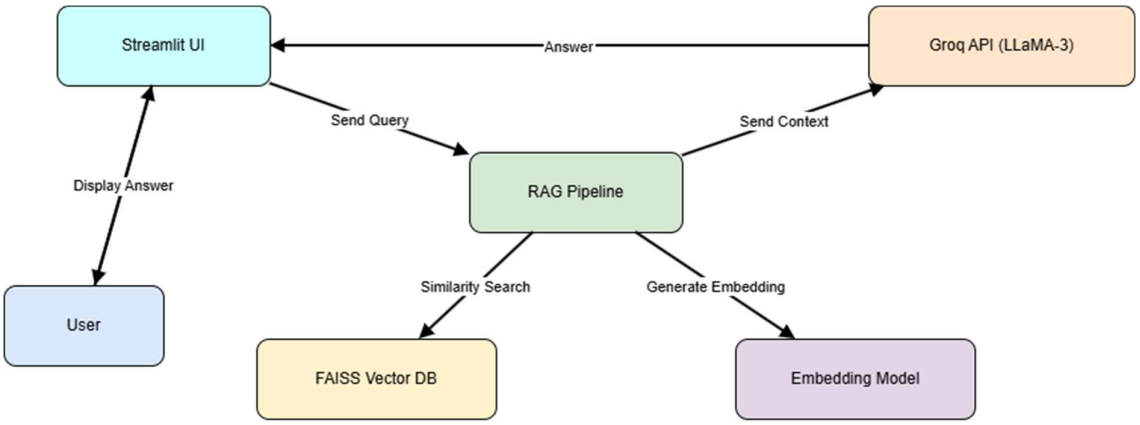


Figure 2.4 - UML Component Diagram

The component diagram illustrates the structural organization of the RAG system across three layers: user interface, application logic, and AI/data services. The backend coordinates document processing and query handling while interacting with the embedding service, vector database, and LLM API. This modular design ensures scalability and clear separation of responsibilities

2.5 Key Design Considerations

1. Scalability

- Stateless API design

- Modular service separation
- Efficient vector indexing

2. Performance

- Optimized chunk size
- Top-K similarity search
- Efficient embedding generation
- Optional caching of frequent queries

3. Security

- Secure API endpoints (HTTPS)
- Authentication & authorization
- PII filtering before LLM call
- Controlled document access

4. Reliability

- Error handling for API failures
- Logging and monitoring
- Retry mechanisms for external API calls

5. Maintainability

- Modular architecture
- Clear separation of responsibilities
- Easy replacement of embedding or LLM models

3. Data Design

The Data Design section defines how data is structured, stored, accessed, retained, and migrated within the RAG-based logistics document intelligence system. The system handles structured, semi-structured, and unstructured data, including documents, embeddings, metadata, and chat sessions.

3.1 Data Model

The system uses a hybrid data model consisting of:

- Relational database (for structured data)
- Vector database (for embeddings)
- File storage (for raw documents)

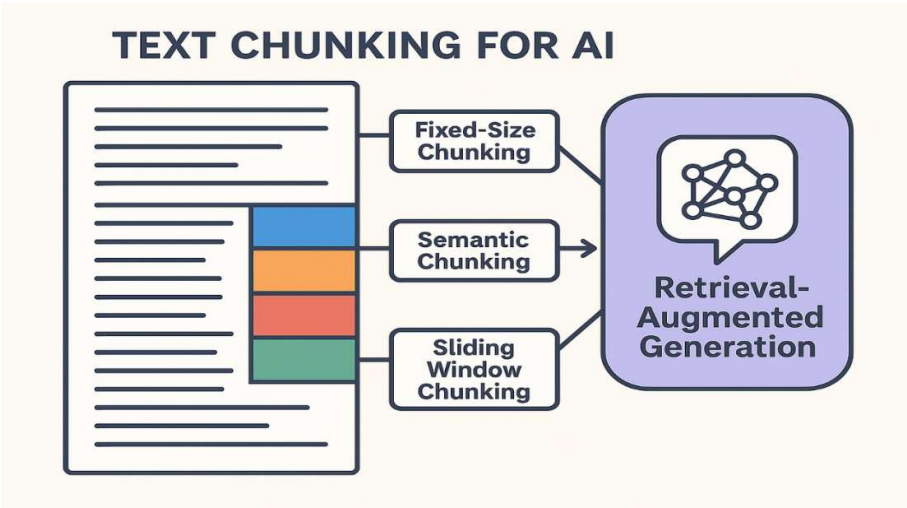


Figure 3.1.1:Chunking Diagram

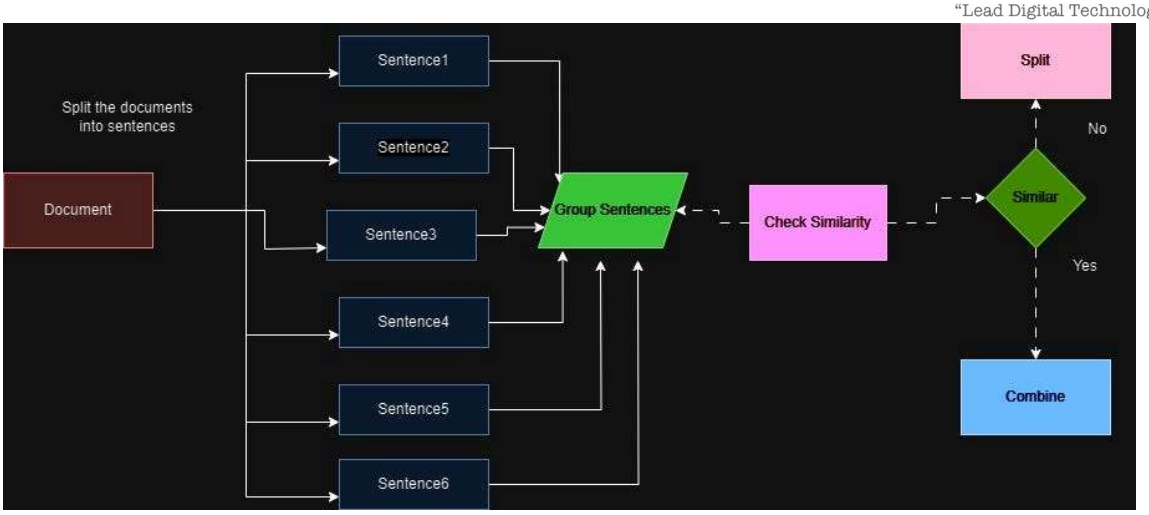


Figure 3.1.2: Split Chunking Diagram

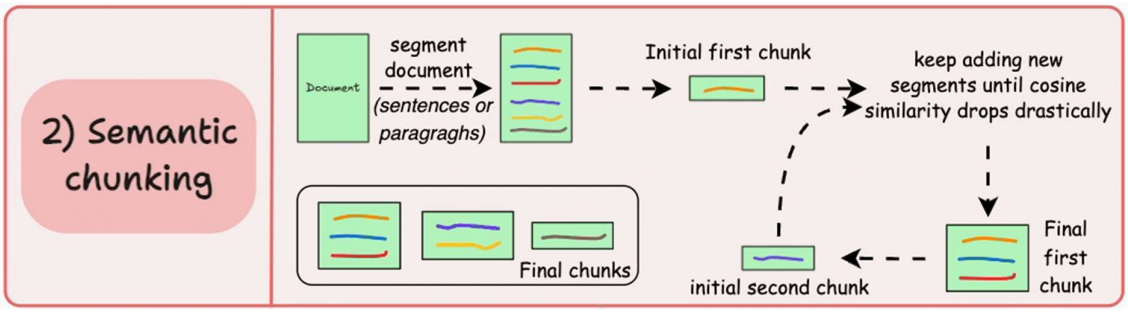


Figure 3.1.3: Semantic Chunking Diagram

Relationships

- One User → Many Documents
- One Document → Many Chunks
- One Session → Many Messages

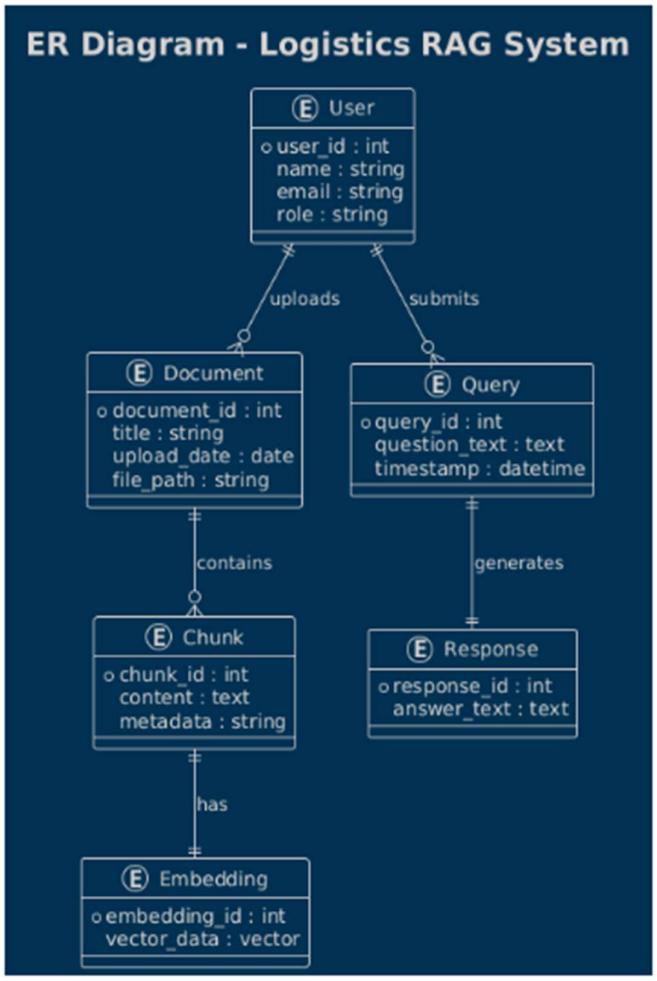


Figure 3.1.4:ER Diagram

The ER diagram represents the core entities involved in the RAG system, including users, documents, chunks, embeddings, queries, and responses. Documents are divided into chunks, each associated with an embedding stored in the vector database. Users upload documents and submit queries, which generate context-based responses.

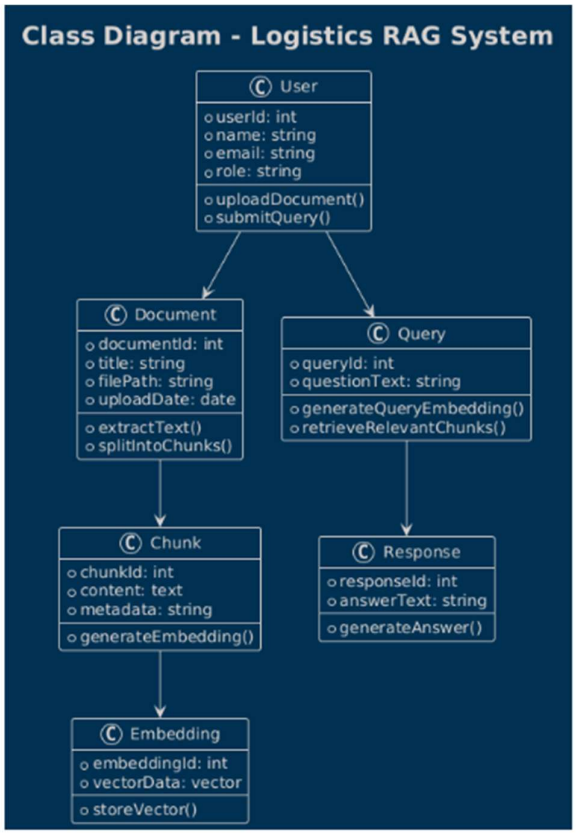


Figure 3.1.5: Class Diagram

The class diagram defines the main system classes such as User, Document, Chunk, Embedding, Query, and Response along with their attributes and methods. Documents are processed into chunks and embeddings, while queries retrieve relevant data to generate responses. The relationships show how each component interacts within the RAG workflow.

3.2 Data Access Mechanism

The system ensures secure and efficient data access through layered access control mechanisms.

1. Structured Data Access

- Stored in relational database (e.g., PostgreSQL).
- Accessed using ORM or SQL queries.
- Indexed fields for faster retrieval.
- Role-Based Access Control (RBAC).

2. Unstructured Data Access

- PDF documents stored in object storage or local storage.

- Accessed via secure file path references.
- Access restricted to authorized users.

3. Vector Data Access

- Stored in FAISS vector database.
- Accessed using similarity search.
- Top-K nearest neighbor retrieval.
- Optional metadata filtering.

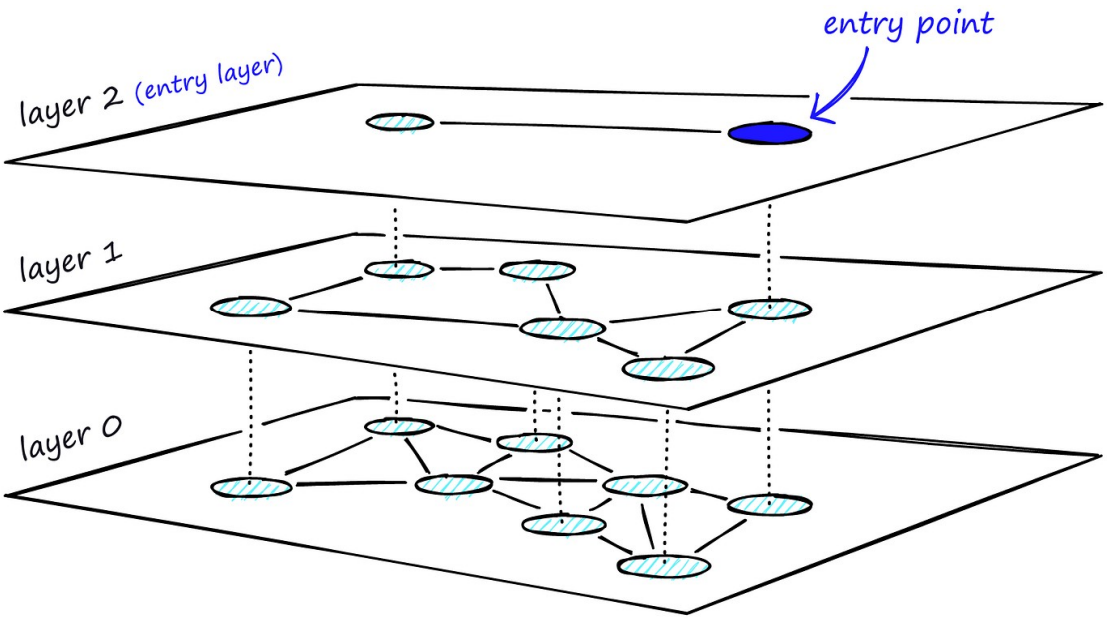


Figure 3.2.1: Vector database diagram

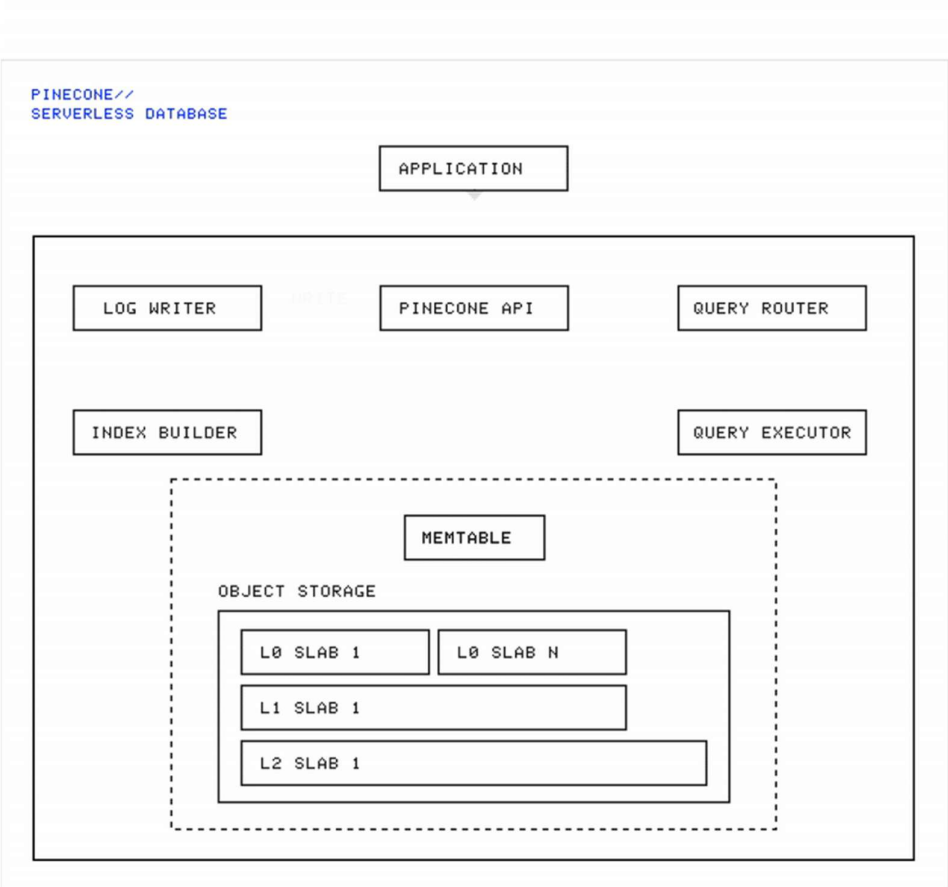


Figure 3.3.2: Serviceless Database

Security Controls

- HTTPS communication
- Token-based authentication (JWT/session ID)
- Input validation and sanitization
- Controlled API access only

3.3 Data Retention Policies

The system defines policies to manage data lifecycle efficiently and securely.

1. Document Retention Policy

- Documents retained for configurable period (e.g., 1 year).
- Users can manually delete documents.
- Upon deletion:

- File removed from storage.
- Embeddings removed from vector database.
- Metadata deleted from relational database.

2. Chat History Retention

- Chat sessions retained for 90 days (configurable).
- Automatic purging after expiry.
- Admin override option available.

3. Backup Policy

- Daily incremental backups.
- Weekly full database backup.
- Backup encryption enabled.

4. Compliance and Security

- Encryption at rest.
- Encryption in transit (HTTPS).
- PII filtering before storing or sending to LLM.
- Logs retained for monitoring and auditing.

3.4 Data Migration

Data migration ensures smooth system upgrades without data loss or service interruption.

1. Schema Migration

- Version-controlled database schema.
- Migration scripts for updates.
- Backward compatibility maintained.

2. Embedding Model Upgrade Migration

If embedding model changes:

- Existing documents are reprocessed.
- New embeddings generated.
- Old embeddings safely replaced.
- Re-indexing performed in batch mode.

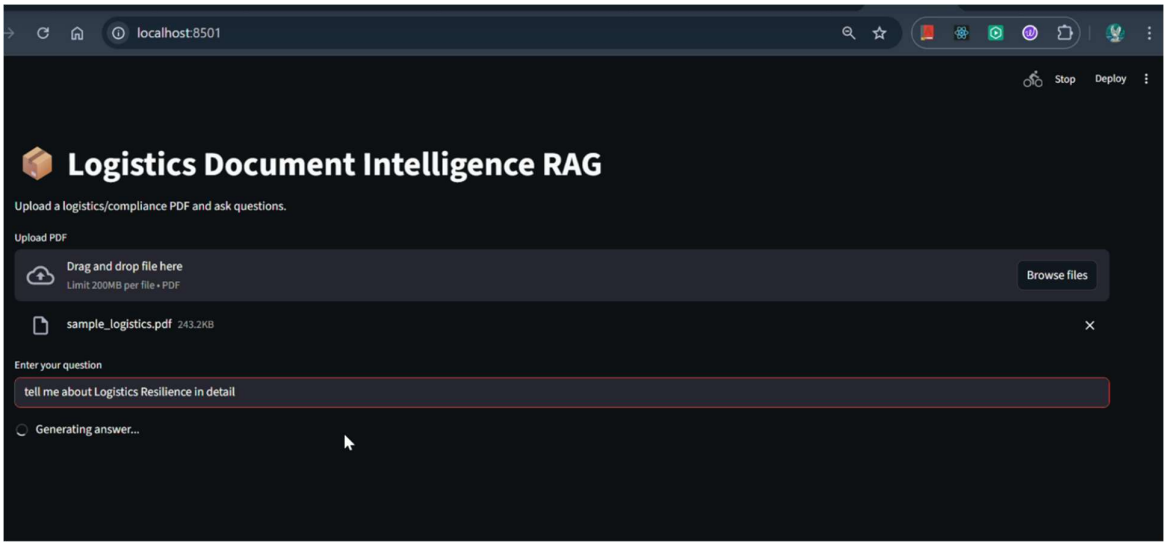
3. Infrastructure Migration

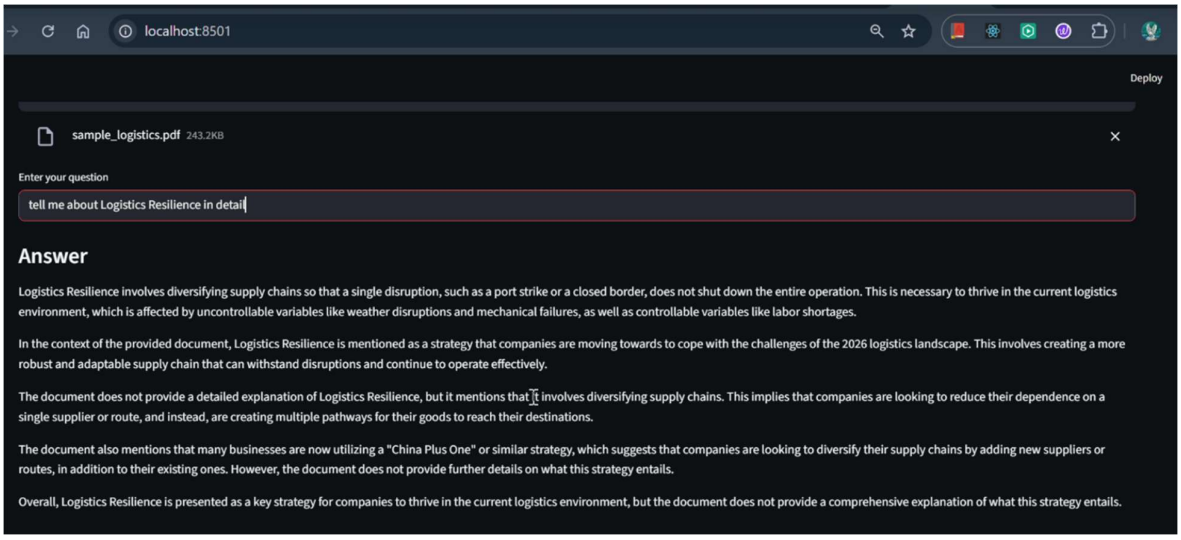
- Data export in structured format (JSON/SQL dump).
- Re-import into upgraded environment.
- Validation and consistency checks performed.

4. Zero-Downtime Strategy

- Blue-green deployment approach.
- Temporary dual indexing during migration.
- Final switch after verification.

4.Interface:





4.1 User Interface (UI)

The primary point of interaction for logistics personnel to manage documents and query the system.

- **Document Upload Interface:** A drag-and-drop or file-explorer interface allowing users to upload professional logistics documents like PDF reports and manuals.
- **Chat Interface:** A natural language dialogue window where users submit queries (e.g., "What is the net weight on Invoice #123?") and receive grounded, context-aware responses with source citations.
- **Document Management Dashboard:** An interface to view, delete, or re-index previously uploaded documents within the knowledge base.

4.2 API Interfaces

The system utilizes several critical APIs to manage the Retrieval-Augmented Generation (RAG) pipeline:

- **Document Processing API (Internal):** A Python-based interface that receives raw PDFs, extracts text, and triggers the chunking and embedding logic.
- **Embedding API (External):** Connects to an LLM provider (e.g., OpenAI, Hugging Face) to convert text chunks into high-dimensional vector representations.
- **Vector Database API:** Interfaces with a database like Pinecone or FAISS to perform high-performance similarity searches and store document vectors.
- **LLM Generation API (External):** Sends the user query combined with retrieved logistics context to a Large Language Model (e.g., GPT-4) to generate a response

  grounded in the provided content.

4.3 Integration Interfaces

- **Data Connector Interface:** Standardized connectors to pull raw material from diverse sources, such as local file systems, cloud drives, or specialized logistics management software.
- **Metadata Interface:** Facilitates the exchange of key document attributes (e.g., Carrier Name, Date, Shipment ID) between the document parser and the vector store to enable advanced filtering during retrieval.

4.4 Error & Exception Handling Interface

- **Validation Filters:** Rule-based interfaces that check for missing or corrupt files during upload and ensure that generated responses do not contain hallucinations or sensitive PII (Personally Identifiable Information).
- **System Status Interface:** Provides real-time feedback on the progress of document indexing or LLM API availability.

5. State and Session Management

5.1 Session Management

Session management ensures that each user interaction is securely tracked during login and document query sessions.

Session Handling Approach

- JWT (JSON Web Token) based authentication.
- Token generated after successful login.
- Token stored securely (HTTP-only cookies or secure storage).
- Token expiry with refresh mechanism.
- Logout invalidates session token.

Session Lifecycle

1. User Login → Token Generated
2. Token Attached to Each API Request
3. Server Validates Token
4. Session Expires After Timeout or Logout

Security Measures

- Token expiration time (e.g., 30–60 minutes)
- HTTPS enforced
- Role-based access control (Admin/User)

5.2 State Management

The system follows a **stateless backend architecture**.

- Each request contains authentication credentials.
- No server-side session storage.
- User state (documents uploaded, query history) stored in database.
- Frontend manages temporary UI states.

6. Caching

Caching improves system performance and reduces response time.

6.1 Types of Caching Used

1. Query Result Caching

- Frequently asked questions cached.
- Reduces repeated LLM calls.

2. Embedding Caching

- Document embeddings stored in vector database.
- Prevents re-processing of same documents.

3. API Response Caching

- Cache frequent metadata requests.
- TTL (Time-To-Live) configuration applied.

6.2 Tools Used

- Redis (for temporary caching)
- In-memory caching for small-scale deployment

7. Non-Functional Requirements

7.1 Security Aspects

Authentication & Authorization

- JWT-based authentication
- Role-based access control

Data Protection

- HTTPS encryption
- Encrypted storage for sensitive data
- Secure API endpoints

Document Security

- User-specific document isolation
- Access control validation before retrieval

Compliance

- Logging and audit tracking
- Secure file upload validation (PDF only)

7.2 Performance Aspects

Scalability

- Modular microservice architecture
- Vector database optimized for large datasets
- Horizontal scaling supported

Response Time

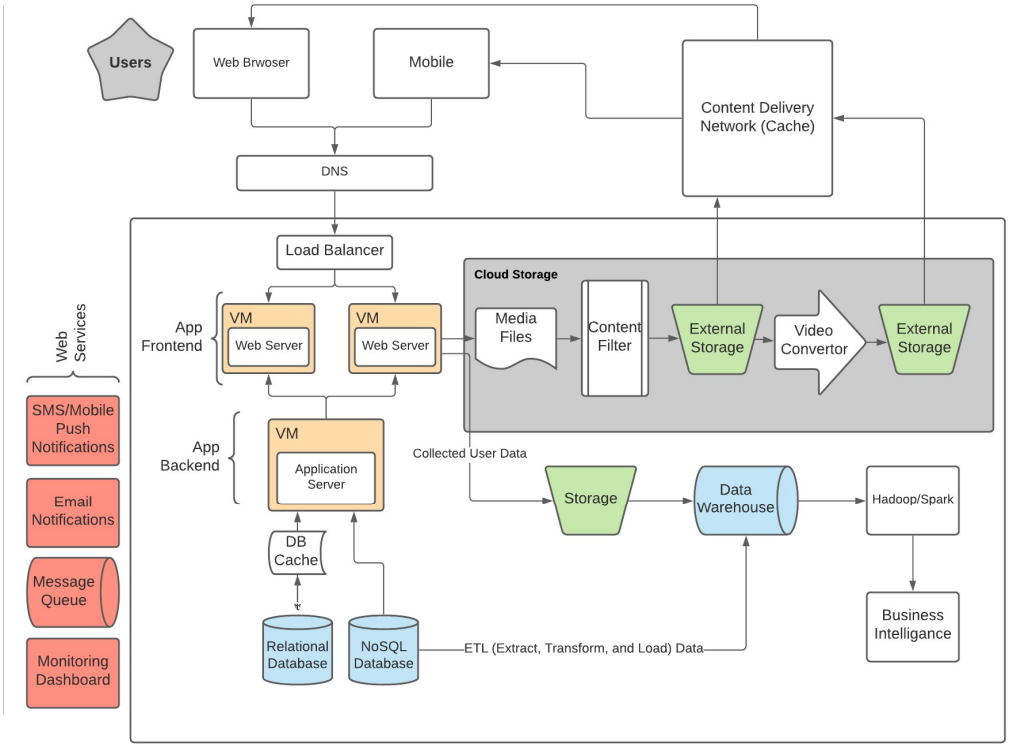
- Target API response time < 3 seconds
- Optimized embedding search

Load Handling

- Asynchronous document processing
- Queue-based processing system

Availability

- 99% uptime target
- Backup and recovery mechanisms



8. References

1. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020.
2. OpenAI API Documentation – LLM Integration
3. FAISS Documentation – Vector Similarity Search
4. LangChain Framework Documentation
5. OWASP Security Guidelines
6. REST API Design Best Practices